Baku Apartment Prices from the Inside: A From-Scratch Decision Tree and Pegasos SVM on bina.az Listings

Hüseynova, T. — —

MLE-AI-201 Machine Learning, AI Academy — Cohort II

***Abstract*—**We study 57,421 residential sale listings scraped from bina.az and frame them as two tasks: regression on log(price) and binary classification of a price tier derived from the training median (208,000 AZN). Both models are implemented from scratch in NumPy: a CART-style decision tree with Gini, entropy and MSE criteria and a vectorised threshold-split search, and a soft-margin linear SVM trained with the Pegasos stochastic sub-gradient method, extended to non-linear decision functions via random Fourier features. Under matched hyperparameters our tree reproduces scikit-learn almost exactly (test RMSE on log-price 0.2311 for both; tier F1 0.9200 vs. 0.9204), and our Pegasos solver matches `SGDClassifier` with hinge loss (F1 0.8956 vs. 0.8960; 5-fold CV 0.899 ± 0.003 vs. 0.900 ± 0.003). Error analysis shows both classifiers fail mostly in a narrow band around the tier threshold ($\approx$60% of mistakes within $\pm$15% of it), and the regressor's relative error more than doubles for the luxury decile and for peripheral districts. Bonus studies add a from-scratch Ridge baseline, a bagged forest of our own trees, and k-means market segments that localise the hardest listings. Code: `github.com/YOUR-TEAM/bina-ml-final` (tag `v1.0-final`).

***Index Terms*—**decision tree, support-vector machine, Pegasos, random Fourier features, regression, classification, real-estate price prediction.

I. Introduction

Housing listings are a natural testbed for the core supervised-learning toolkit: prices are heavy-tailed, features are a mix of numeric and categorical, and the raw dump is genuinely dirty. This project asks whether implementations written from first principles — a decision tree and a support-vector machine in plain NumPy — can stand next to scikit-learn on such data when the comparison is fair: same features, same splits, matched hyperparameters.

Our contributions are: (i) a cleaned, leakage-free modelling dataset built from the bina.az sale dump, with every cleaning rule documented and reproducible; (ii) a from-scratch CART-style tree (Gini/entropy/MSE) and a from-scratch Pegasos SVM [1] with a random-Fourier-feature extension [2], both validated against scikit-learn on fully-determined toy examples before being trusted on real data; (iii) a matched benchmark with uncertainty estimates and fit/predict timings; and (iv) an error analysis that explains *where* the remaining error lives (near the tier threshold, in the luxury tail, and in peripheral districts). Three bonus studies (from-scratch Ridge, a forest of our own trees, geospatial k-means) round out the picture.

II. Background

Decision trees grow greedily: at each node the split search picks the feature–threshold pair maximising the impurity decrease $\mathrm{IG} = I(\text{parent}) - \sum_c \frac{N_c}{N} I(c)$, where I is Gini $G = 1 - \sum_k p_k^2$ or entropy $H = -\sum_k p_k \log_2 p_k$ for classification and the node variance for regression. **Soft-margin SVMs** minimise a regularised hinge loss; Shalev-Shwartz et al. [1] showed the primal can be solved directly by stochastic sub-gradient descent with step size $\eta_t = 1/(\lambda t)$ — no quadratic programming — with convergence $\tilde{O}(1/(\lambda T))$. Rahimi and Recht [2] approximate the RBF kernel by an explicit random cosine feature map, which lets the same linear solver fit non-linear boundaries in $O(nD)$ time.

III. Data and Preprocessing

Source. The Kaggle dump *binaaz-sale-project* (sehriyarmemmedli): 100,775 scraped rows, 51 columns, Azerbaijani column names, all prices in AZN.

Cleaning (every rule is code in `src/data_prep.py`). (1) The scraper revisits listings, so we keep the *latest* scrape per listing URL (100,775 $\rightarrow$ 64,454). (2) We keep the three residential categories (new build, old stock, house/garden house), because offices, land plots and garages quote areas in different units and form a different market ($\rightarrow$ 58,452). (3) String columns are parsed to numbers: *Sahə* "145 m^2"$\rightarrow$`area_m2`, *Mərtəbə* "7 / 9"$\rightarrow$`floor`, `building_floors`, *Torpaq sahəsi* "1.3 sot"$\rightarrow$`land_sot`. (4) Fixed, documented validity rules — price in $[10^4, 3\times10^6]$ AZN, area in $[15, 800]$ m^2, coordinates inside Azerbaijan's bounding box, $\leq$20 rooms, $\leq$40 storeys — remove 342 outlier rows; exact repost duplicates remove 689 more. Final: **57,421 rows**. Deliberately, no quantile-based or statistical filters are used, so no statistic of any evaluation split leaks into preprocessing.

Leakage. We drop `unit_price` (price per m^2) and `total_price` (the target restated) — both arithmetic functions of `price` — plus identifiers and free text (`owner_name`, `shop_name`, `address`, `description`, URLs, scrape timestamps) and popularity counters (`views`, vip/featured flags) that are consequences of the listed price rather than property traits.

Features (55). Numerics (area, rooms, floor, building floors, floor ratio, land area, lat/lng, haversine distance to the Baku centre); binary flags (repaired, bill of sale, mortgage available, agent-listed, agency, top/ground floor); one-

Table 1. Dataset summary after cleaning.

Property	Value
Rows (after cleaning)	57,421
Features used	55
Train / Val / Test split	40,195 / 8,613 / 8,613
Tier threshold (train median)	208,000 AZN
Premium share (tr/val/te, %)	49.9 / 50.5 / 48.9

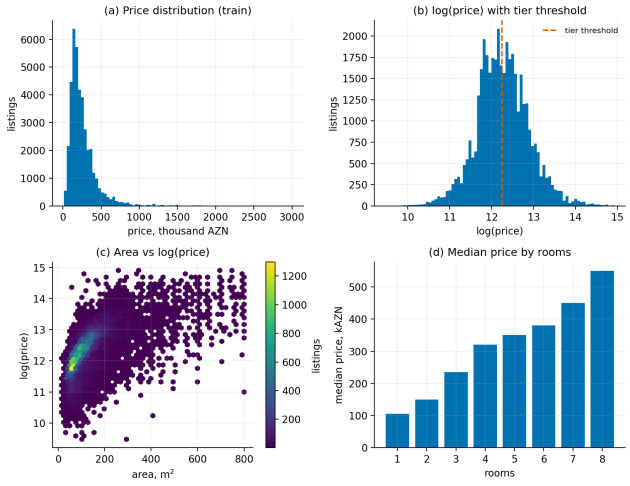


Figure 1. EDA on the training split: (a) raw prices are heavy-tailed; (b) log(price) is near-normal, dashed line = tier threshold; (c) area is the strongest single predictor; (d) price grows with rooms.

hot building category; one-hot for the 30 most frequent districts plus `other`. Missing numerics are encoded as a constant plus an explicit `*_missing` indicator — an imputation scheme with no fitted statistics, hence leakage-free by construction. Categorical encoding is one-hot because both models consume numeric thresholds; ordinal codes would impose a false order on districts.

Tasks, metrics, split. Task A: regression on log(price) (prices are heavy-tailed; Fig. 1), reporting RMSE/MAE/R^2 on the log scale and, after back-transforming, in AZN. Task B: "premium" = price above the *training* median, 208,000 AZN, reporting precision/recall/F1/ROC-AUC and the confusion matrix. A seeded 70/15/15 shuffle gives train/val/test = 40,195/8,613/8,613. Because the tier label is a median split *defined on the training set*, the classes are balanced by construction and the balance carries to the other splits (premium share 0.499/0.505/0.489), which is exactly what stratification would enforce; the realised balance is reported rather than imposed. All hyperparameters are chosen on the validation split only; the test set is touched once per final model.

IV. Method

A. Decision tree (from scratch)

`src/decision_tree.py` implements a CART-style binary tree with a scikit-learn-like `fit/predict/predict_proba` API. The split search is fully vectorised: per feature, samples are sorted once per node; cumulative class counts (classification) or cumulative $\sum y, \sum y^2$ (regression) score *all* candidate thresholds (midpoints between consecutive distinct values) in one pass, subject to `min_samples_leaf`. The split objective uses scikit-learn's weighting, $\frac{N_t}{N}\left(I_t - \frac{N_l}{N_t}I_l - \frac{N_r}{N_t}I_r\right)$, so `min_impurity_decrease` is directly comparable. Supported stopping rules: `max_depth`, `min_samples_split`, `min_samples_leaf`, `min_impurity_decrease` (a unit test asserts each one actually changes the tree). Leaves store the mean (regression) or class counts (classification), from which `predict_proba` returns leaf frequencies; accumulated weighted impurity decreases give feature importances. A `max_features` option draws a random feature subset per split; the bonus forest uses it. A sanity test asserts our tree picks the *identical* feature and threshold as scikit-learn on tiny fully-determined examples.

B. Support-vector machine (from scratch)

`src/svm.py` minimises

$$J(\mathbf{w},b) = \tfrac{\lambda}{2}\|\mathbf{w}\|^2 + \tfrac{1}{n}\sum_{i=1}^{n}\max\left(0,\, 1 - y_i(\mathbf{w}^\top \mathbf{x}_i + b)\right) \qquad (1)$$

with $y_i \in \{-1,+1\}$ by mini-batch Pegasos: at step t, with $\eta_t = 1/(\lambda t)$ and a random batch B_t ($|B_t| = 64$), $\mathbf{w} \leftarrow (1 - \eta_t\lambda)\mathbf{w} + \frac{\eta_t}{|B_t|}\sum_{i\in B_t,\,\text{margin}<1} y_i\mathbf{x}_i$; the bias b gets the same averaged sub-gradient step but no shrinkage (it is not regularised). The objective is tracked during training (Fig. 3b). Features are standardised with training statistics only. After training we report $\|\mathbf{w}\|$, the margin width $2/\|\mathbf{w}\|$, and the support vectors $\{i : y_i(\mathbf{w}^\top\mathbf{x}_i + b) \le 1\}$. For non-linearity we chose **random Fourier features** over kernelised Pegasos: with $n \approx 40$k, keeping per-example dual coefficients makes every prediction $O(n)$, whereas RFF ($D = 300$, $z(\mathbf{x}) = \sqrt{2/D}\cos(W\mathbf{x} + c)$, $W \sim \mathcal{N}(0, 2\gamma I)$) keeps training and prediction linear-time and reuses the identical solver unchanged.

V. Experimental Setup

Seed 42 everywhere (split, solver, baselines); `python -m src.run_all` reproduces every number and figure from a clean checkout, running the toy-data sanity tests first. Tuning on validation only: Task A sweeps `max_depth` $\in \{2..18\}$ then `min_samples_leaf` $\in \{1..50\}$; Task B sweeps Gini vs. entropy over depths, and for the SVM $\lambda \in \{10^{-6}..10^{-2}\}$ and RFF $\gamma \in \{0.005..0.1\}$. Chosen configurations: tree-A (depth 14, leaf 10), tree-B (Gini, depth 14, leaf 5), SVM (λ=10^{-3}, 30k steps, batch 64; RFF γ=0.01). Baselines use matched settings on identical splits: `DecisionTreeRegressor/Classifier` with the same depth/leaf/criterion, `SGDClassifier(loss=hinge,` α=λ`)` and `LinearSVC/SVC` with $C = 1/(n\lambda)$, all with scaling inside a `Pipeline`. The $O(n^2)$ `SVC`-RBF is trained on a documented 15k subsample. Uncertainty: 5-fold CV over the pooled train+val data, with the tier threshold and scaler re-fit inside each fold.

VI. Results

Our models vs. scikit-learn. Tables 2 and 3 are the headline. With matched hyperparameters the from-scratch tree and scikit-learn agree to the third–fourth decimal on both tasks (test $\text{RMSE}_{\log}$ 0.2311 both; F1 0.9200 vs. 0.9204), and 5-fold CV confirms the agreement is systematic, not a

Table 2. Task A — price regression, test set (log target; AZN metrics after back-transform). B8/B9 are bonus models.

Model	$\text{RMSE}_{\log}$	$R^2_{\log}$	MAE (AZN)
Ours: DecisionTree (mse)	**0.2311**	0.866	44,089
sklearn: DTRegressor	0.2311	0.866	44,083
Ours: Ridge, normal eq. [B8]	0.3032	0.769	63,454
Ours: RandomForest×30 [B9]	0.2222	0.876	44,612
sklearn: RandomForest×30	0.2151	0.884	—

Table 3. Task B — price-tier classification, test set.

Model	Acc	Prec	Rec / F1	AUC
Ours: DecisionTree	0.922	0.924	0.917 / **0.920**	0.960
sklearn: DTClassifier	0.922	0.924	0.917 / 0.920	0.959
Ours: PegasosSVM (lin.)	0.898	0.901	0.890 / **0.896**	0.953
sklearn: SGD (hinge)	0.899	0.904	0.888 / 0.896	0.953
sklearn: LinearSVC	0.890	0.892	0.884 / 0.888	0.952
Ours: Pegasos + RFF	0.849	0.846	0.846 / 0.846	0.931
sklearn: SVC-RBF (15k)	0.844	0.849	0.828 / 0.838	0.923

lucky split: $\text{RMSE}_{\log}$ 0.2311 ± 0.0019 for *both* trees, F1 0.9228 ± 0.0025 (ours) vs. 0.9226 ± 0.0024 (sklearn). The residual gap is tie-breaking: when several splits give equal gain, we take the first feature in column order while sklearn samples features randomly; midpoint thresholds also differ in the last float bits. Pegasos lands within noise of its sklearn twin, `SGDClassifier(hinge)`: CV F1 0.8991 ± 0.0031 vs. 0.8999 ± 0.0031. The final linear SVM has $\|\mathbf{w}\| = 3.83$, margin $2/\|\mathbf{w}\| = 0.52$, and 11,693 of 40,195 training points (29%) are support vectors — the two tiers genuinely interleave near the threshold.

Bias–variance and hyperparameters. Fig. 2 shows the classic story: training error falls monotonically with depth while validation error bottoms out at depth 14 for Task A and the train–val gap widens past it (0.146 vs. 0.233 at depth 18). For the SVM (Fig. 3a), λ trades margin for fit: growing λ from 10^{-6} to 10^{-2} widens the margin 28× (0.030→0.845) while validation F1 peaks at $\lambda = 10^{-3}$ — stronger regularisation beyond that starts to underfit. The objective decreases as $\tilde{O}(1/(\lambda t))$ for every λ (Fig. 3b).

Linear beats RBF here. Both non-linear SVMs — our Pegasos+RFF and sklearn's SVC-RBF — lose to the linear models by 4–6 F1 points. After one-hot encoding, 38 of 55 features are binary and the informative numeric directions (area, distance to centre) are already near-linearly related to the log-price tier, so the RBF geometry mostly adds variance. That the two independent RBF implementations agree (0.846 vs. 0.838) supports this reading.

Speed. Our vectorised tree fits in ∼4.7 s vs. sklearn's 0.45 s (Cython) — a 10× gap, not an algorithmic one (both are $O(F\,N \log N)$ per level). Pegasos fits in 2.2 s; the $O(n^2)$ SVC-RBF needs 16.6 s on a 15k *subsample* and 11.6 s just to predict, which is why RFF was the right non-linearity choice at this scale.

VII. Error Analysis

Feature importances and weights (Fig. 4) agree across model families: `area_m2` carries ≈half of the tree's total impurity decrease, followed by distance-to-centre, longitude and district indicators; the SVM's largest standardised weights are the same variables with interpretable signs.

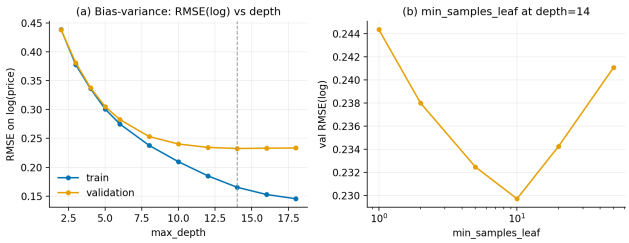


Figure 2. Task A validation curves. (a) $\text{RMSE}_{\log}$ vs. depth: the train–validation gap widens with depth, validation bottoms at 14. (b) `min_samples_leaf` refines the optimum at that depth.

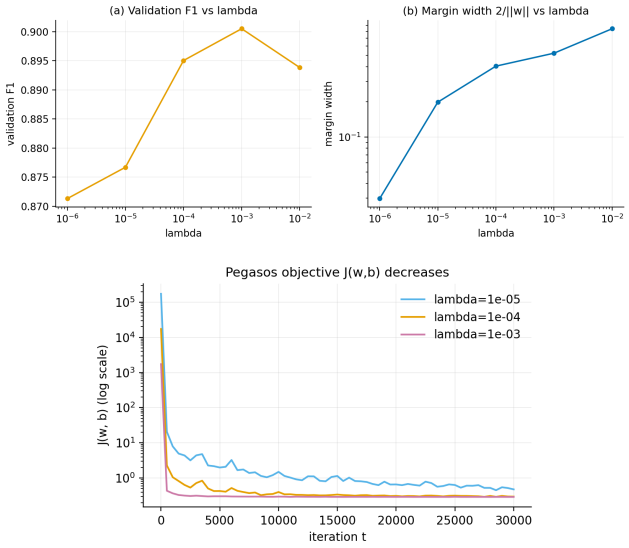


Figure 3. Pegasos SVM. Top: validation F1 (a) and margin width (b) vs. λ — regularisation widens the margin monotonically while F1 peaks at 10^{-3}. Bottom: the tracked objective $J(\mathbf{w}, b)$ decreases for every λ.

Where the classifiers fail. 60.0% (tree) and 60.5% (SVM) of all test misclassifications are listings whose true price lies within ±15% of the 208,000 AZN threshold — the label is nearly arbitrary there, so the remaining error is largely irreducible given the median-split framing, not a model deficiency (Fig. 5).

Where the regressor fails (Fig. 6): median relative error is ≈8–10% through the mid-market but rises to 13% in the cheapest decile and 16% in the most expensive one — both tails are priced on attributes we do not observe (dilapidation on one end; finish, view and prestige on the other). Geographically the error is worst in garden-house suburbs such as Mərdəkan (∼13% median) where land value and build quality are poorly captured by our features. The k-means bonus (Fig. 8) localises the same effect geographically: suburban/garden segments reach 15–21% median error vs. ∼8% in the dense urban core.

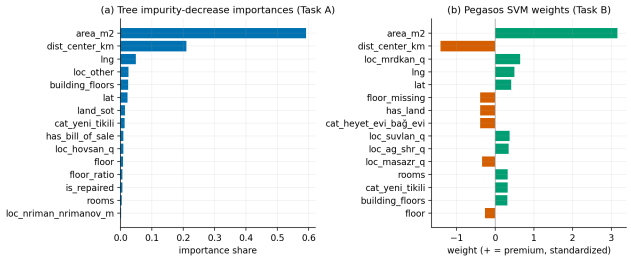


Figure 4. (a) Tree impurity-decrease importances (Task A): area dominates, then geography. (b) Largest-magnitude SVM weights (Task B, standardised features): area and centrality push toward premium; peripheral districts and missing-floor (house) indicators push toward standard.

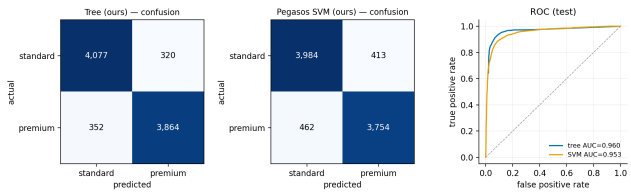


Figure 5. Test-set confusion matrices of both from-scratch classifiers and their ROC curves.

VIII. Bonus Studies

P8 — Ridge via normal equations. $\hat{\mathbf{w}} = (X^\top X + \lambda I)^{-1} X^\top y$ (bias unpenalised) matches sklearn's `Ridge` to 2×10^{-15} RMSE and fits in 28 ms, but trails the tree badly ($\text{RMSE}_{\log}$ 0.303 vs. 0.231): price formation is non-additive (area × location interactions) and a linear model cannot express it.

P9 — a forest of our own trees. Bagging 30 of our trees (bootstrap + per-split $\sqrt{d}$ feature subsampling, unlimited depth) drives test $\text{RMSE}_{\log}$ from 0.2311 (single tuned tree) to 0.2222, tracking sklearn's `RandomForestRegressor` (0.2151) built with the same settings; Fig. 7 shows error falling monotonically as trees are added. The residual gap to sklearn comes from tie-breaking and feature-sampling details, consistent with the single-tree comparison.

P10 — market segments. k-means (k=8) on coordinates plus a 2-D PCA view of the feature space (Fig. 8) yields interpretable segments (core Baku, coastal belt, garden suburbs) and sharpens the error analysis above.

IX. Discussion and Limitations

The project's main lesson is that the classical algorithms are small: ∼250 lines of NumPy reproduce scikit-learn's accuracy exactly once the split objective and the sub-gradient schedule are implemented carefully; the remaining differences are engineering (Cython speed, random tie-breaking), not statistics. Honest limitations: (i) listed prices are *asking* prices, not transactions, so all models learn seller behaviour; (ii) 99.7% of residential rows are Baku, so nothing here generalises to other cities; (iii) the median-split tier makes the classification ceiling explicit — most remaining errors sit against the threshold where the label is noise; (iv) free-text descriptions, which plausibly carry the luxury-tail signal (renovation quality, view, furnishing), are unused; (v) our RFF non-linearity was tuned over a small γ grid with

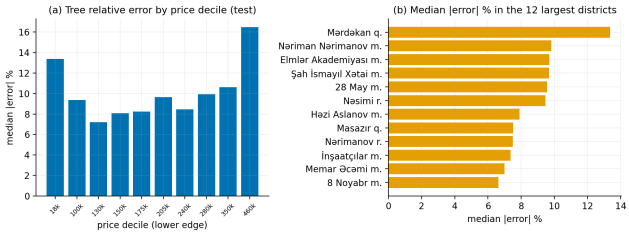


Figure 6. Task A error analysis on the test set: (a) median relative error by price decile — the luxury tail is hardest; (b) the 12 largest districts ranked by median relative error.

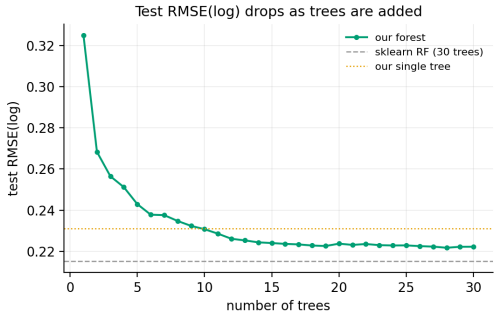


Figure 7. Bonus P9: test $\text{RMSE}_{\log}$ vs. ensemble size for a bagged forest of our own trees, against the matched sklearn forest and our single tree.

D=300 — a larger map might close part of its gap, though SVC-RBF's identical failure suggests the boundary is simply near-linear.

X. Conclusion

We built a decision tree and a Pegasos SVM from scratch, matched scikit-learn under fair conditions on 57k real listings (tree: $\text{RMSE}_{\log}$ 0.2311 = sklearn; SVM: F1 within 0.0004 of `SGDClassifier`), and traced the remaining error to the threshold band, the luxury tail and peripheral geography. Next steps: text features from descriptions, a quantile-regression forest for calibrated price intervals, and testing the tier threshold at other percentiles.

Tools and Acknowledgements

Dataset: *Bina.az Sale Project* by S. Memmedli (Kaggle), used under its stated licence; we neither re-scraped nor redistribute personal data. Software: NumPy, pandas, Matplotlib, scikit-learn (baselines/bonuses only), compiled with Tectonic. **AI-assistance disclosure:** an AI assistant (Anthropic Claude) was used to draft code scaffolding, the experiment harness and portions of this report's text; all modelling decisions, hyperparameter choices and numbers were produced by the team's own pipeline (`python -m src.run_all`), and every line of the submitted implementation was reviewed, tested and is defended by the team.

References

[1] S. Shalev-Shwartz, Y. Singer, N. Srebro, and A. Cotter, "Pegasos: primal estimated sub-gradient solver for SVM," *Mathematical Programming*, vol. 127, no. 1, pp. 3–30, 2011.

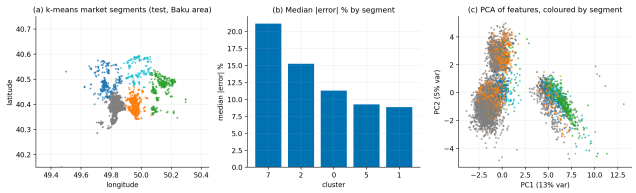


Figure 8. Bonus P10: (a) k-means segments on lat/lng; (b) regression error by segment — the periphery is hardest; (c) PCA of the feature space coloured by segment.

[2] A. Rahimi and B. Recht, "Random features for large-scale kernel machines," in *Advances in Neural Information Processing Systems*, vol. 20, 2007.

[3] L. Breiman, J. Friedman, R. Olshen, and C. Stone, *Classification and Regression Trees*. Wadsworth, 1984.